TIME APPLIANCES
(TAP)



High-Radix Switches and Precise Time

Dan Biederman

Distinguished Technologist, Eridu

OCP STARTUP™ Member

Disclaimer

- This document represent my views on the topic and my not represent the views of Eridu, any standards body, or any specific customer or vendor.

Agenda

- Latency
- High-radix switching and latency
- High-radix switches, precise time and the parking lot problem
- Conclusion
- Call to Action and Q&A

Latency Drives AI Performance

- AI workloads demand low and predictable latency
- If you can remove a microsecond of latency…

Inference Costs ↓

Training Costs ↓

Tokens Per Watt ↑

Potentially Save Billions of Dollars!



DOI:10.1145/3015146

Microsecond-scale I/O means tension between performance and productivity that will need new latency-mitigating ideas, including in hardware.

BY LUIZ BARROSO, MIKE MARTY, DAVID PATTERSON, AND PARTHASARATHY RANGANATHAN

Attack of the Killer Microseconds

THE COMPUTER SYSTEMS we use today make it easy for programmers to mitigate event latencies in the nanosecond and millisecond time scales (such as DRAM accesses at tens or hundreds of nanoseconds and disk I/Os at a few milliseconds) but significantly lack support for microsecond (µs)-scale events. This oversight is quickly becoming a serious problem for programming warehouse-scale computers, where efficient handling of microsecond-scale events is becoming paramount for a new breed of low-latency I/O devices ranging from datacenter networking to emerging memories (see the first sidebar "Is the Microsecond Getting Enough Respect?").

Processor designers have developed multiple techniques to facilitate a deep memory hierarchy that works at the nanosecond scale by providing a simple synchronous programming interface to the memory system. A load operation will logically block a thread's execution, with the program appearing to resume after the load completes. A host of complex microarchitectural techniques make high performance possible while supporting this intuitive programming model. Techniques include prefetching, out-of-order execution, and branch prediction. Since nanosecond-scale devices are so fast, low-level interactions are performed primarily by hardware.

At the other end of the latency-mitigating spectrum, computer scientists have worked on a number of techniques—typically software based—to deal with the millisecond time scale. Operating system context switching is a notable example. For instance, when a `read()` system call to a disk is made, the operating system kicks off the low-level I/O operation but also performs a software context switch to a different thread to make use of the processor during the disk operation. The original thread resumes execution sometime after the I/O completes. The long overhead of making a disk access (milliseconds) easily outweighs the cost of two context switches (microseconds). Millisecond-scale devices are slow enough that the cost of these software-based mechanisms can be amortized (see Table 1).

These synchronous models for interacting with nanosecond- and millisecond-scale devices are easier than the alternative of asynchronous models. In an asynchronous programming model, the program sends a request to a device and continue processing other work

» key insights

- A new breed of low-latency I/O devices, ranging from faster datacenter networking to emerging non-volatile memories and accelerators, motivates greater interest in microsecond-scale latencies.
- Existing system optimizations targeting nanosecond- and millisecond-scale events are inadequate for events in the microsecond range.
- New techniques are needed to enable simple programs to achieve high performance when microsecond-scale latencies are involved, including new microarchitecture support.

ILLUSTRATION BY PETER CROW THIES ASSOCIATES

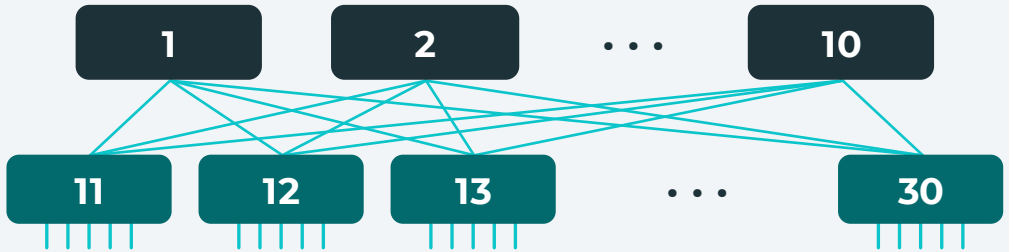
https://dl.acm.org/doi/10.1145/3015146

High-Radix Switches and Latency



High Radix Switching Flattens Networks and Delivers Massive Latency Benefits

30 x 102T switches
640 x 1.6T ports



Hypothetical Single 1P switch
640 x 1.6T ports



Single Hop

Latency Benefits

- Eliminate switch hops → reduce average latency
- Eliminate congestion points, simplify congestion avoidance and control → reduce jitter / tail latency

Benefits of High-Radix Switching

Larger scale-out/across clusters with flatter, simpler networks

Larger high-bandwidth single-hop scale-up domains

Economics
Higher GPU utilization
Lower cost & power

Performance
Faster job completion
Lower latency & congestion

Operations
Higher reliability, simpler cabling, faster deployment

BETTER AI OUTCOMES



More model intelligence & flexibility



Faster training



Lower cost per token

Hypothetical Multi-Hop Latency

- Switch Latency (T):
  - Advertised = X microsecond
  - Realistically it may take ~5X
- Using a high radix switch allows fewer layers,
  - One switch level versus two levels results ~1/3 the latency of a normal switch.
  - This does not include the congestion related impact on latency due to a more complex networks.

Each extra level adds ~10X!

Switch Levels	Advertised Latency	Realistic Latency
1	X	5X
2	3X	15X
3	5X	25X
4	7X	35X
5	9X	45X

High-Radix Switches, Precise Time, and the Parking Lot Problem



High-radix Switches and Precise Time

Characteristics of High-radix Switch

- Hundreds to thousands of ports
- Retain the benefits of a fixed-form-factor switch
  - Configure and operate as a single switch
  - Latency of a single switch -- no hidden hops inside the box

Precise time implications

Precise time needs to be correct at

- Every port
- Every time-aware feature

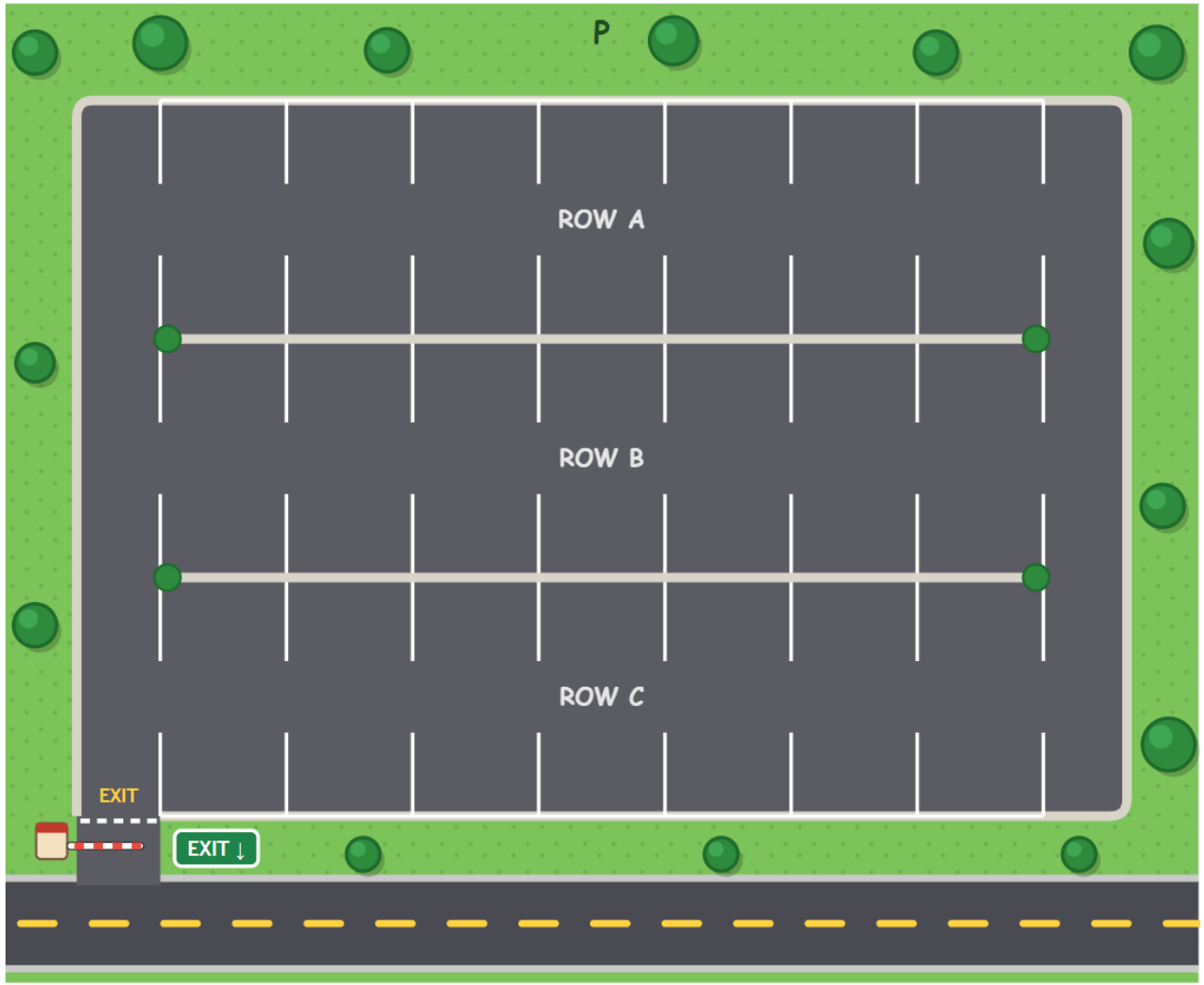
Synchronous Ethernet

A single port's high-quality clock needs to be used for every Ethernet transmit clock

- Thousands of SERDES and their PLLs must be synchronized to the incoming high-quality port's frequency
- Having Sync-E can make the IEEE 1588 timekeeping more precise
- Limitations?

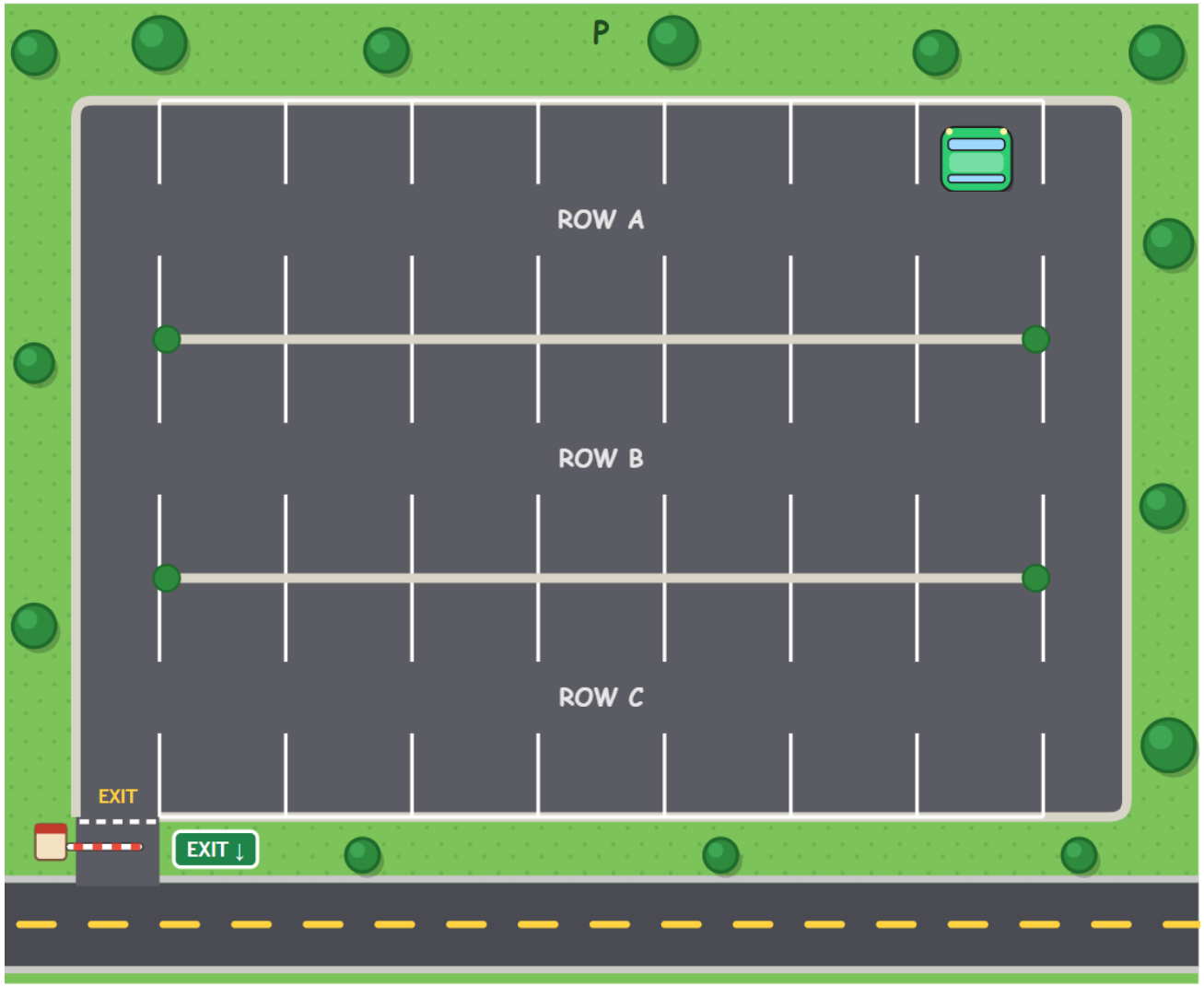
Precise Time Solves the Parking Lot Problem

- The Physical Requirements of Large High Radix Switching is Significant.
  - Many packets arrive for the same output port as each packet travels to the output port
  - Some packets take longer times.
    - Congestion / Path Length
- Parking Lot Problem Helps Explain:
  - The car furthest into the parking lot takes the longest time to exit.
- Solution:
  - Precise Arrival Times at all Inputs
- Fairness Metrics:
  - Arrival/Residency time needs to be considered
  - Result: Reduce Tail Latency



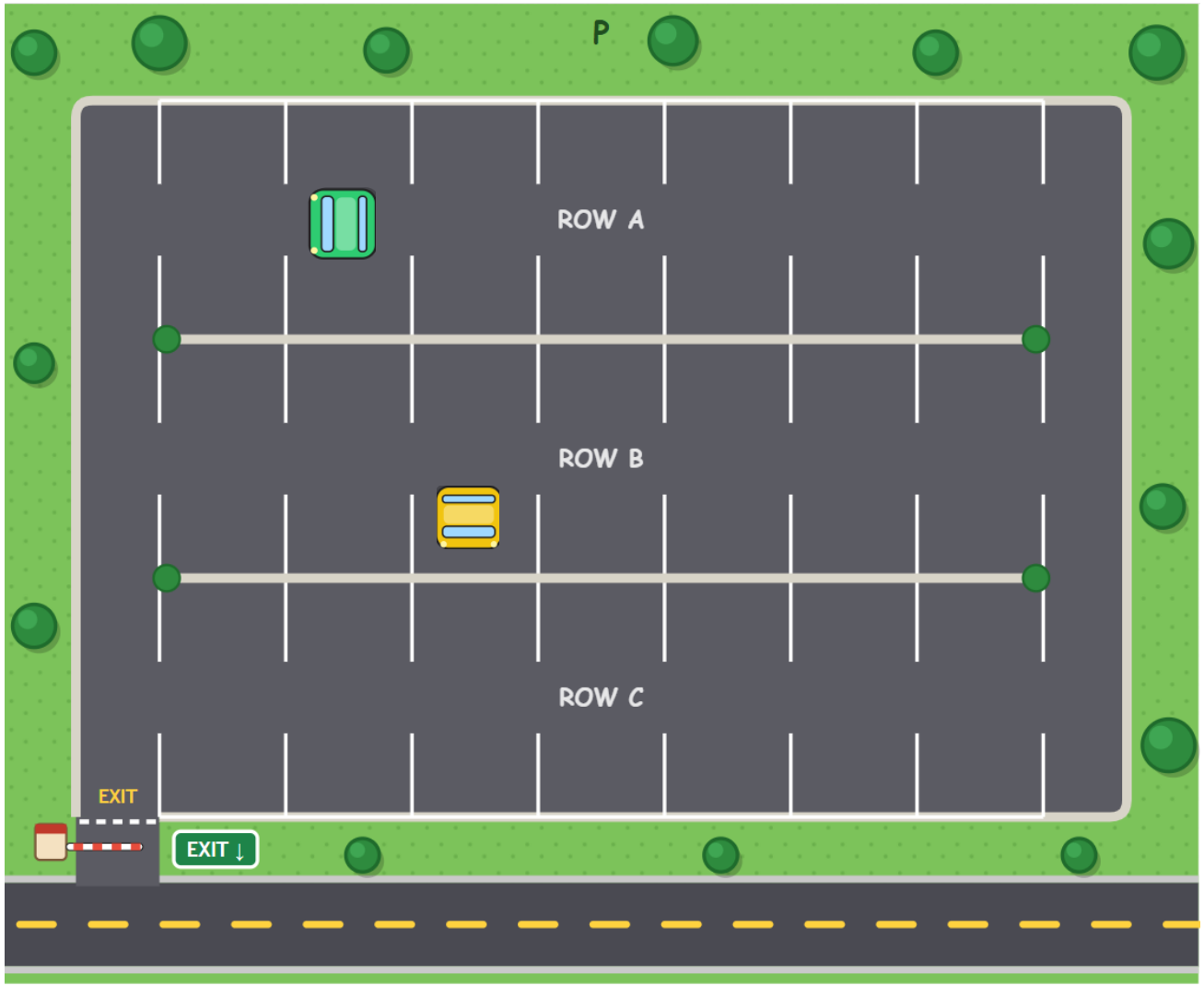
Precise Time Solves the Parking Lot Problem

- Green car wants to leave at time X.



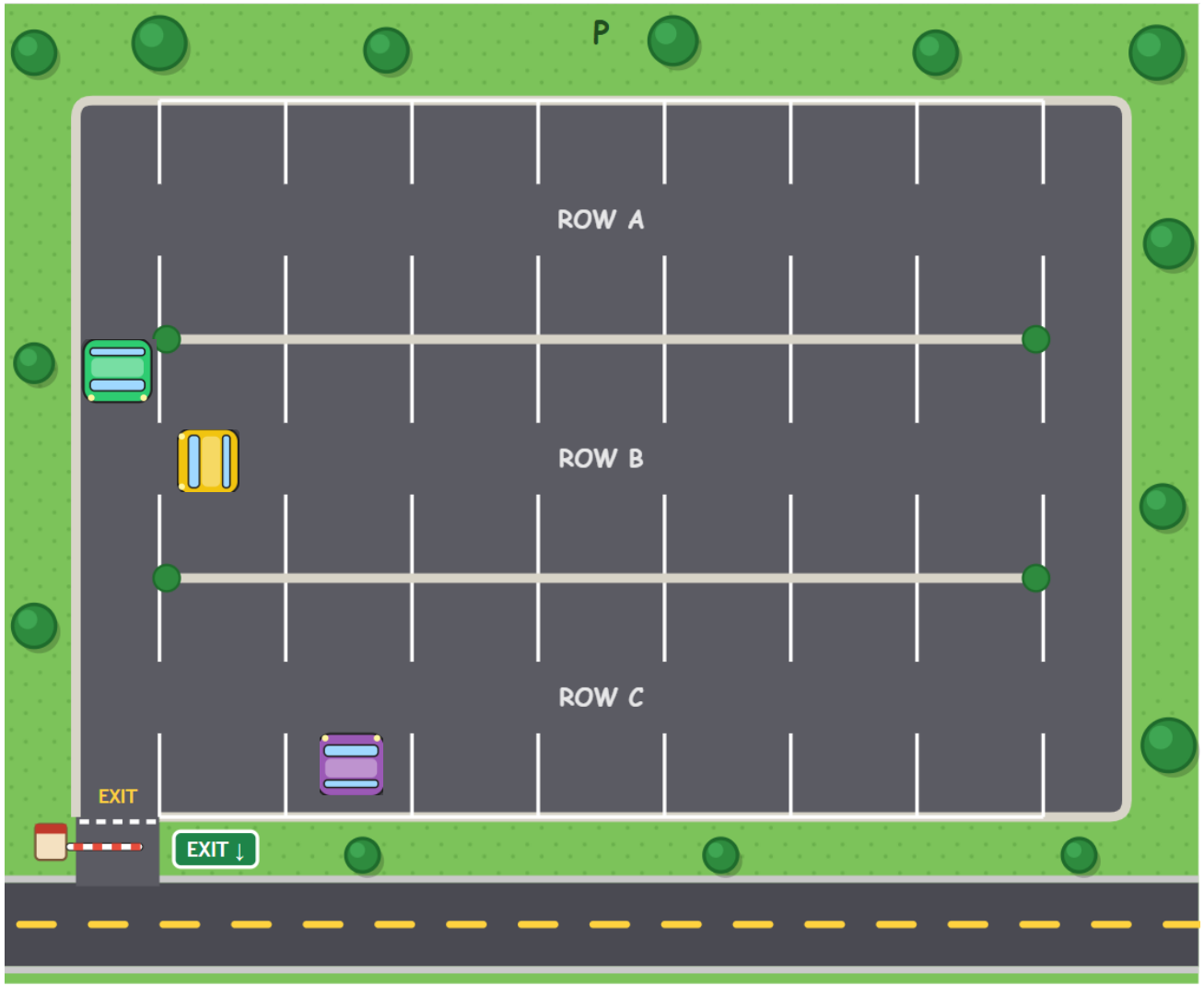
Precise Time Solves the Parking Lot Problem

- Green car wants to leave at time X.
- As the green car is driving towards an exit a yellow car also wants to leave.



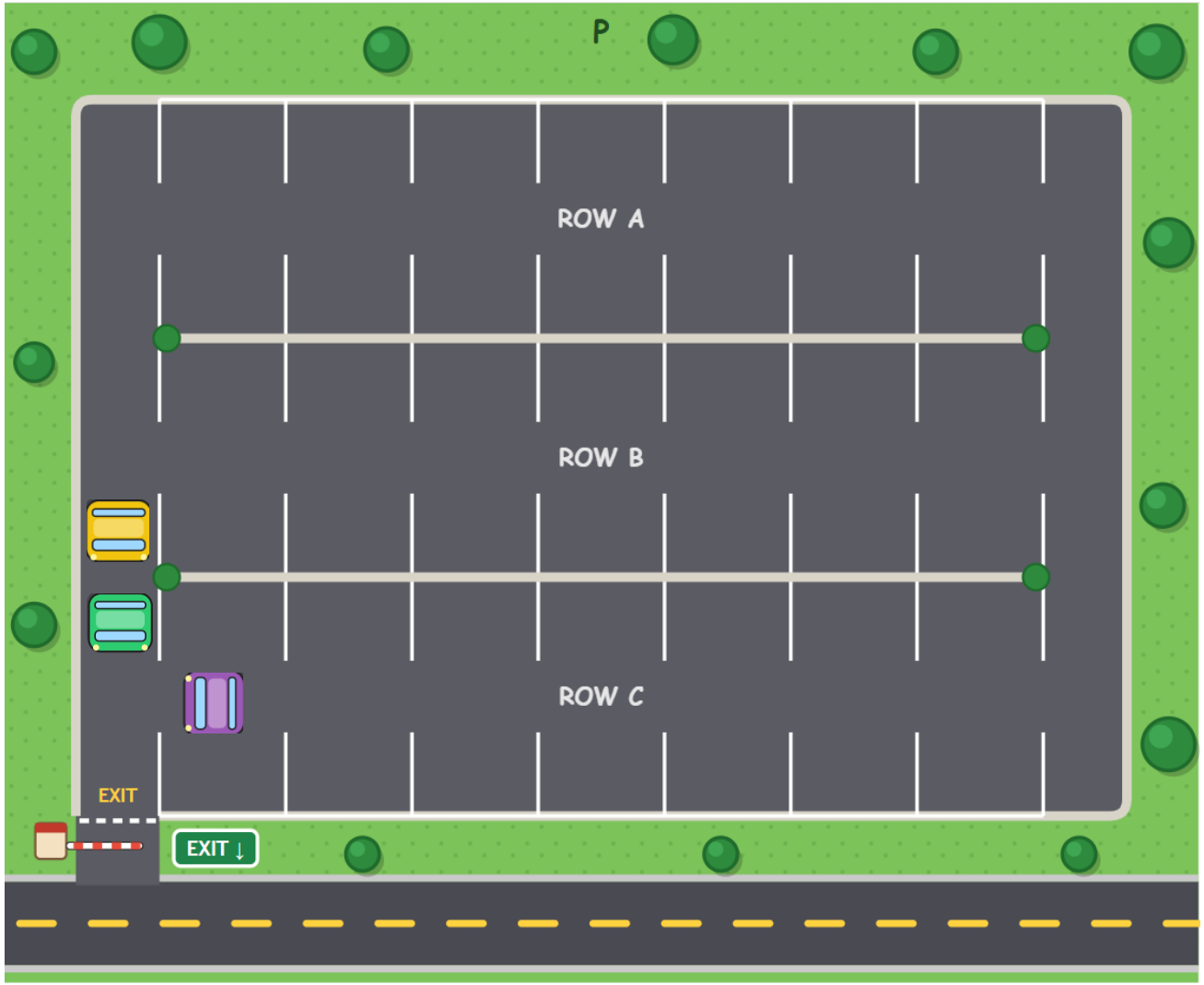
Precise Time Solves the Parking Lot Problem

- Green car wants to leave at time X.
- As the green car is driving towards an exit a yellow car also wants to leave.
- Later a purple car wants to leave
- Only one car can pass at an intersection, the earlier car wins.



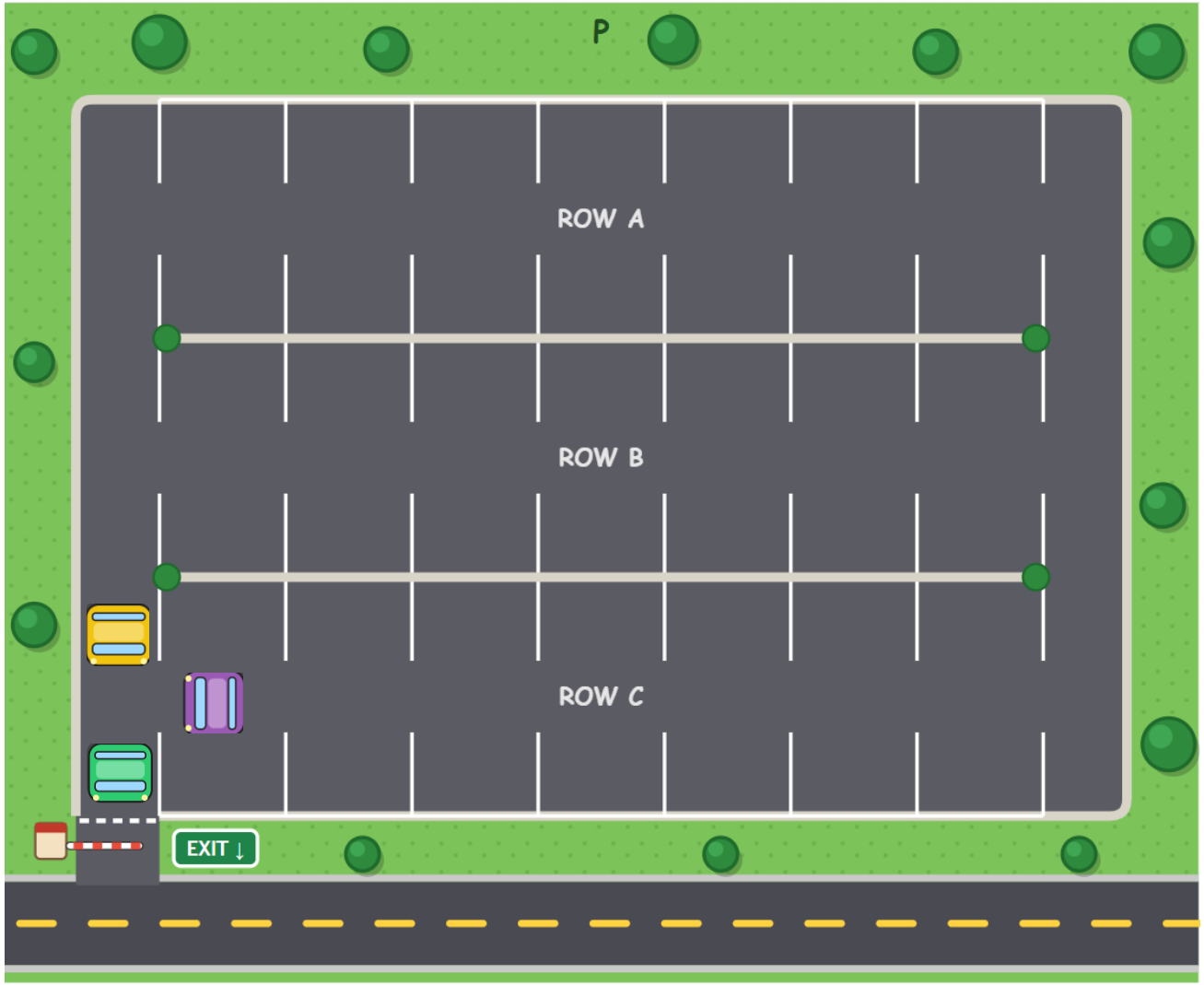
Precise Time Solves the Parking Lot Problem

- Green car wants to leave at time X.
- As the green car is driving towards an exit a yellow car also wants to leave.
- Later a purple car wants to leave
- Only one car can pass at an intersection, the earlier car wins.
- Green started first, and wins.



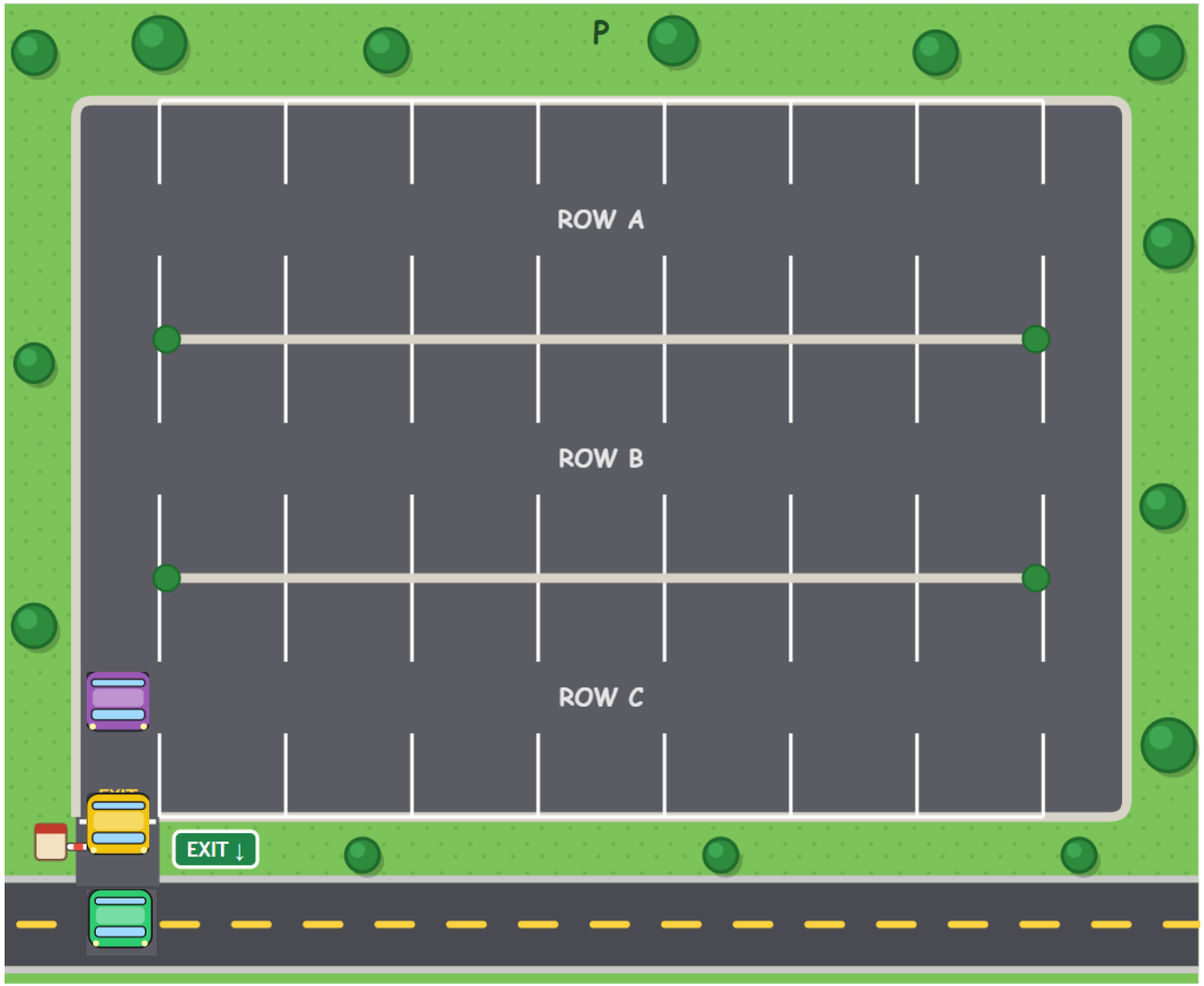
Precise Time Solves the Parking Lot Problem

- Green car wants to leave at time X.
- As the green car is driving towards an exit a yellow car also wants to leave.
- Later a purple car wants to leave
- Only one car can pass at an intersection, the earlier car wins.
- Green started first, and wins.
- Yellow was second and wins next.



Precise Time Solves the Parking Lot Problem

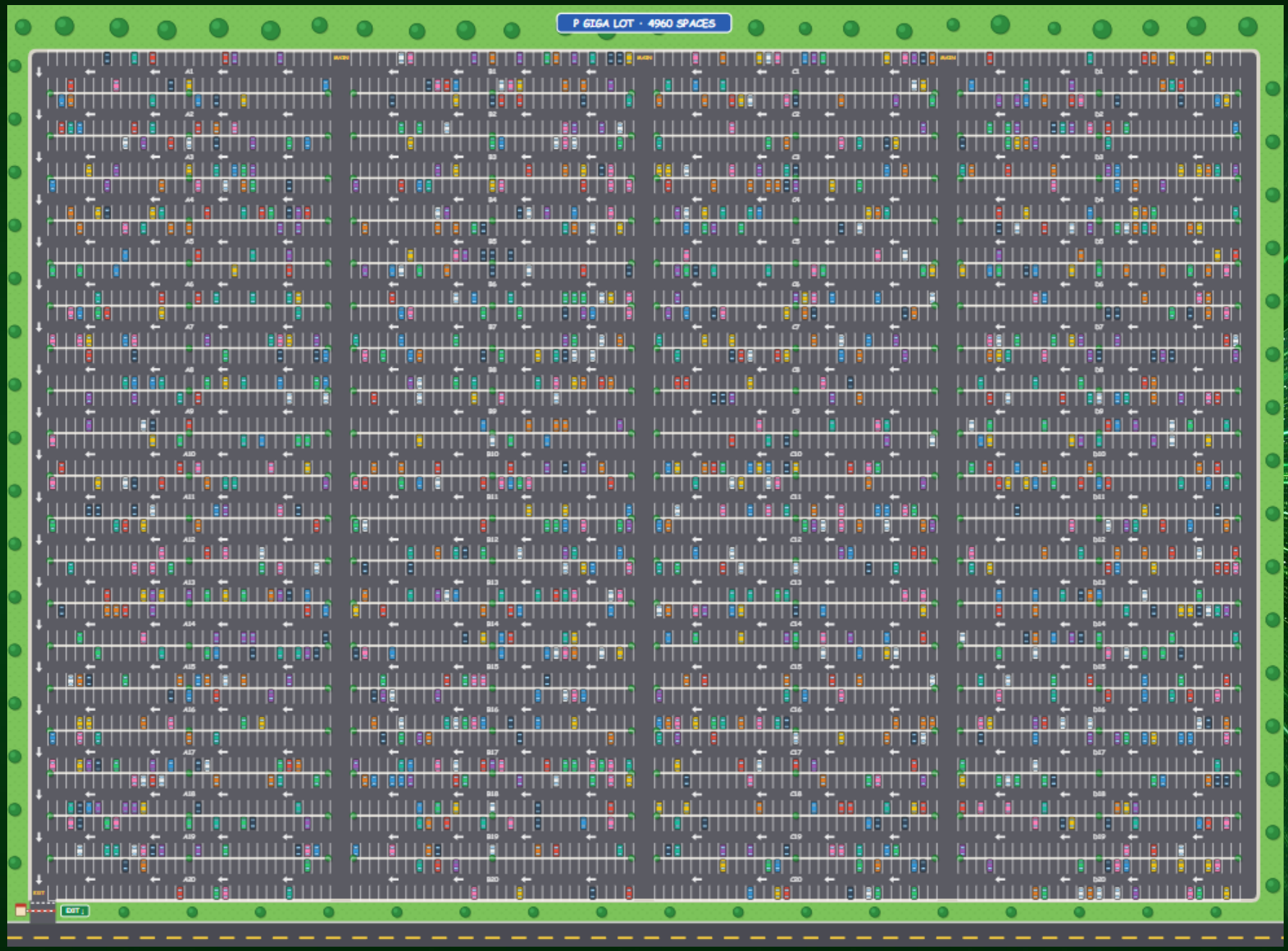
- Green car wants to leave at time X.
- As the green car is driving towards an exit a yellow car also wants to leave.
- Later a purple car wants to leave
- Only one car can pass at an intersection, the earlier car wins.
- Green started first, and wins.
- Yellow was second and wins next.
- Purple is last.



Now Consider the Problem with Over 1000 Spaces

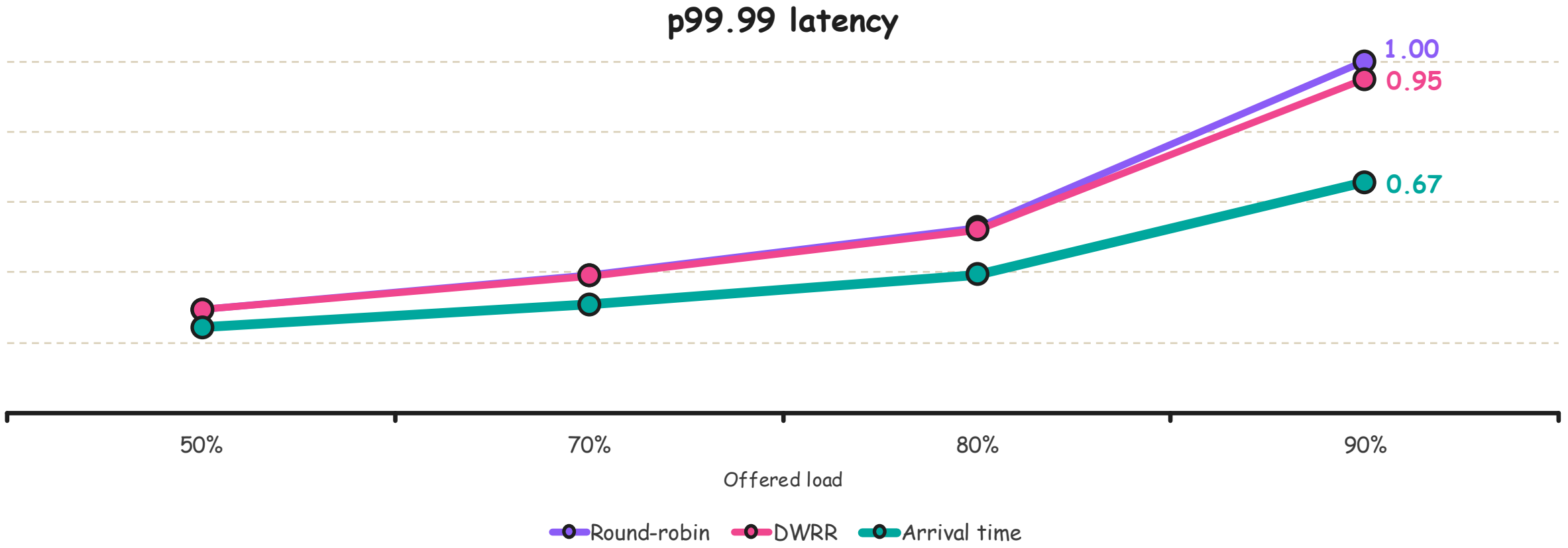


~5000 Spaces



Ideal Arrival-Time Scheduler Has the Shortest Tail

High Radix Simulated Results.



P99.99: Arrival time vs DWRR: **–17% at 50% load, –31% at 90% load**

1000×1000 switch, arrival-time scheduling study, mean of 100 Monte Carlo runs per load.

Conclusions

- ✓ High-radix switching reduces network latency & jitter
  - Flattens networks, minimize hops → larger single-hop domains in scale-up and scale-out
  - Significantly reduces network latency and jitter versus lower-radix switches

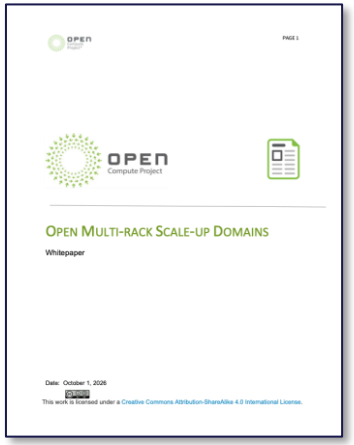
  (And the benefits of high radix go beyond latency: power, cost, reliability, GPU utilization, training/inference cost, etc.)

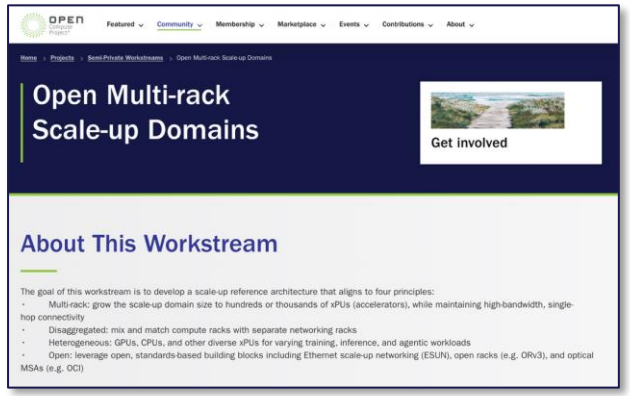
- ✓ Precise time helps manage and contain latency thru the high-radix switches
  - Precise Time Measurement allows visibility into network latencies.
  - Sync-E enables high-quality clock distribution
  - Precise time enables solutions to address the parking lot problem

Call to Action

1. Help drive Precise Time in the data center and avoid those “Killer Microseconds”

2. Help drive Open Multi-Rack Scale-up Domains (new OCP workstream)





https://www.opencompute.org/projects/semi-private-workstreams/open-multi-rack-scale-up-domains/


Thank
You!
OCP
GLOBAL
SUMMIT
October 12-15, 2026
San Jose, California